



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

CS7CS4 Machine Learning

Week 2

Shane O'Donnell – 21364002
Dataset ID: 14-14--14

Introduction

Supervised learning techniques are an immensely important in the current rise of Artificial Intelligence and Machine Learning techniques. This exploration was split into three distinct parts and entailed utilising a binary classification dataset to visualise the dataset, train a logistic regression classifier and analyse its parameters, predictions and decision boundary created. Secondly by training a linear support vector machine (SVM) with different penalty values, allowed an investigation into the effect of using different regularisation strengths affects a model's parameters, predictions and decision boundary. Finally, the set of features provided was extended to four by including the squared terms of the original features, allowing the model to capture nonlinear behaviour. This model's accuracy was then compared to the original logistic regression model.

The learning outcomes evident in this investigation were to develop training, evaluating and visualising models by using the necessary python libraries and increase ones understanding of how feature engineering can improve performance in models.

Method

In order to begin utilising the dataset provided, the data was downloaded from the link provided and stored as a csv file. In this format the data could then be read and stored appropriately for the exercise by separately storing both features $[X_1, X_2]$ and the corresponding output $[y]$. To prepare the data further for visualisation, the feature components were split to two groups depending on their output being positive or negative. Once this was complete it was possible to visualise the data expressing their feature values and the true label of the data point. This graph can be seen below in figure 1.

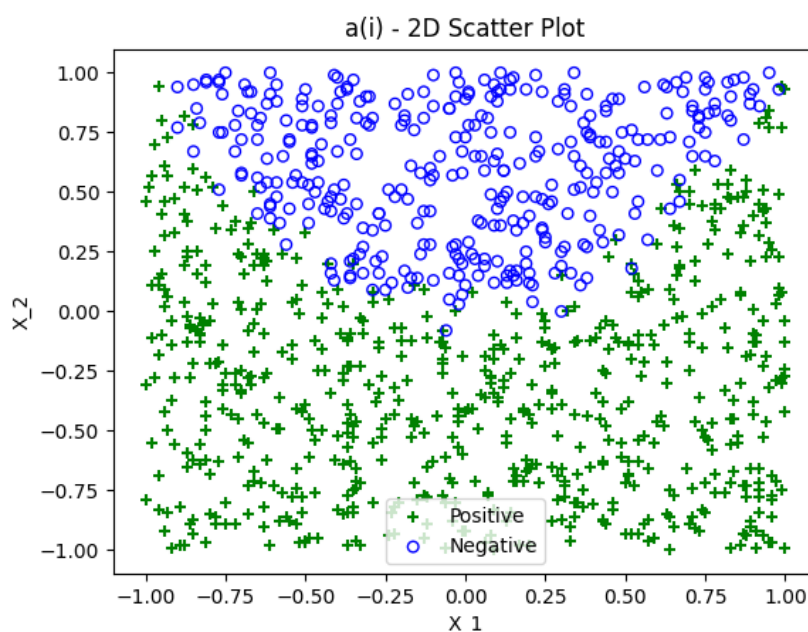


Figure 1: Dataset visualised by feature pair values and grouped by true output

To begin training and testing models, it was necessary to split the dataset into “test” and “train” sections. The split chosen to conduct the objectives outlined in the brief was 80:20, this was chosen to allow the model a significant cut of the data to train and learn on and also allow enough data to obtain a realistic metric of accuracy for the model.

Once this dataset provided had been grouped and visualised as described above, it was possible to train a logistic regression classifier model. For this exercise, the loss function chose was L2 loss with a liblinear solver; these specifications were chosen as according to the sklearn documentation for the “LinearRegression” tool, L2 is the default choice and unlike L1, L2 (ridge) does not set features to 0 which will allow all features will retain influence. Furthermore, according to the documentation “liblinear” is optimal for small datasets. As well as this an inverse regularisation strength chosen was 1 as this offers a standard strength of inverse regularisation. The model trained to identify the decision boundary for this exercise can be displayed using the following formula shown below, the x variables represent the features, the w 's represent the weights identified by training the model and the b value being the intercept of the decision boundary line.

$$x_1w_1 + x_2w_2 + b$$

To further this the probability of a set of features corresponding to a positive value can be calculated with the following formula.

$$p(y = 1|x) = \sigma(x_1w_1 + x_2w_2 + b)$$

$$p(y = 1|x) = \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}$$

In order to visualise the overall accuracy of the model trained on the provided data, the true data was mapped as well as the test data predictions from the trained model. As well as this, the boundary line was visualised to display how inaccuracies can occur most prominently near the boundary line. This visualisation can be seen below in figure 2. As seen in the plot, some points on the graph can be seen visualised as black plus signs, this is due to an overlay of “Predicted Negative” points and “True Positive” points; due to the conflicting colours the points appear as a dark plus sign. This overlay highlights how this model falls short as it can not capture the nonlinear behaviour of the model meaning the decision boundary mapped leads to many mispredictions. The models function can be seen below. From the weights associated with the model, it can be inferred that the x_1 feature has very little influence on the model while the x_2 feature is most influential, this can be seen on the plot in figure 2 by the fact that the proposed boundary splits the y axis and has minimal slope.

$$p(y = 1|x) = 0.090358x_1 - 4.897536x_2 + 1.6176379$$

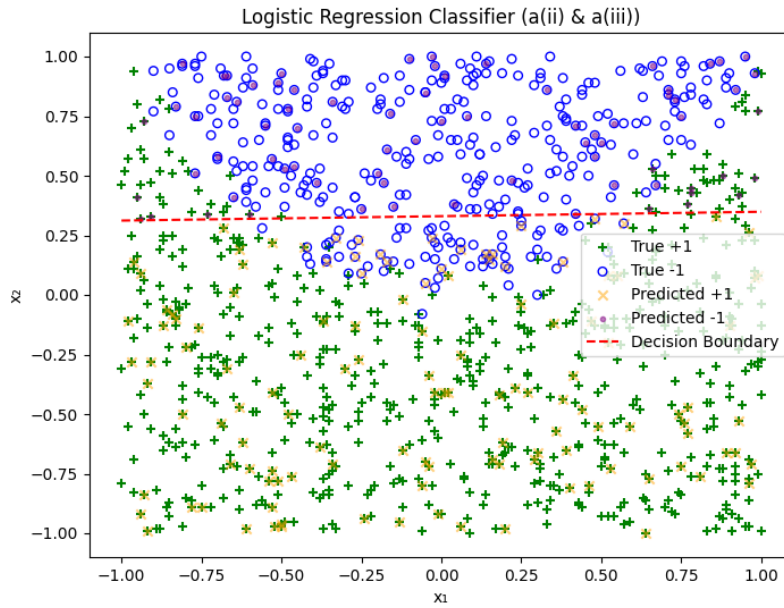


Figure 2: Logistic Regression Classifier with True points, Predictions and decision boundary

The second exploration in this investigation entailed using a linear support vector machine (SVM) to model the same data utilised for the first investigation. While both a logistic regression classifier model and SVM model both produce a linear boundary, they differ in their implementation; while Logistic Regression places the decision boundary where the model predicts a 50% chance of correctly predicting the label, a SVM model is placed to maximise the gap between two classes which often causes the model to ignore points that fall far away from the margin. As well as this a logistic regression model models probability, the formula for this can be seen above in investigation one. Support Vector Machines only focuses on separating classes with the widest margin possible which can be seen in the formula below. Hinge loss is used to penalise points which fall on the wrong side of the boundary.

$$\min \frac{1}{2} \|\omega\|^2 + C \sum \text{hinge loss}$$

In order to further analyse SVM applications, multiple SVM models were trained with different specifications. Figure 3 below outlines the possible parameters provided for SVM's using sklearn's LinearSVC library.

Parameter	Description
penalty	This specifies the norm used in the penalisation, the default is L2 but also supports L1
Loss	Specifies the loss function used, hinge is the standard for SVM loss but squared_hinge is also supported
C	Regularisation parameter. The strength of the regularisation is inversely proportional to C.
max_iter	This specifies the maximum number of iterations to be run

Figure 3: Possible Parameters of LinearSVC from sklearn library

The first investigation entails isolating the C parameters and analysing the effect on the model whilst other parameters and the dataset stay constant. In order to conduct this analysis a for loop was used to iterate through the same parameters except for changing the C value to [0.001,1,100]. The first model visualised can be seen in figure 4 which described the model with a regularisation parameter of 0.001. From the visualisation it is possible to analyse that due to a small C value, hinge loss is minimised due to this corresponding to high regularisation. This causes the model to prioritise a larger minimum distance between the boundary and the closest data points from each class. Comparatively to the logistic regression decision boundary, this strong regularisation pushes the decision boundary higher and causes the model to prioritise this wider margin instead of fitting the points closer to the true class boundary.

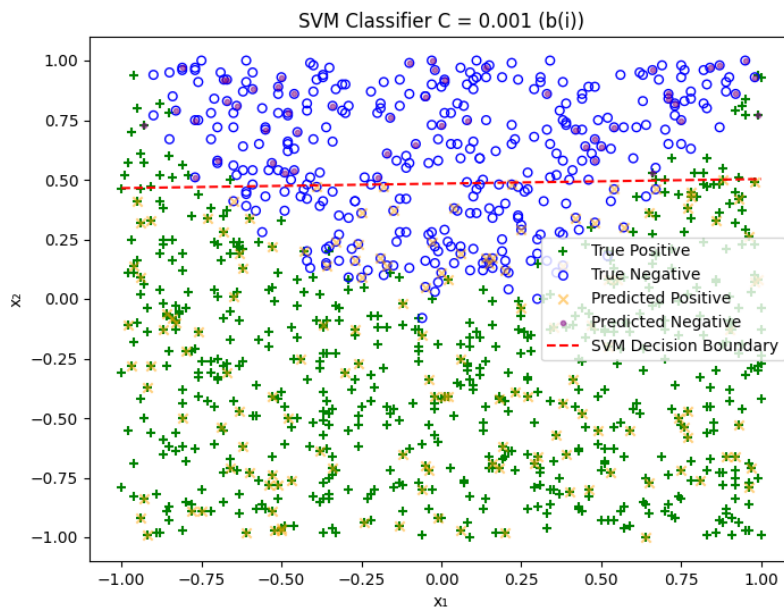


Figure 4: SVM Classifier with a C (regularisation) of 0.001

From the training process the model's decision boundary can be described mathematically as seen below with the x variables representing the two features present. The weight attached to the first feature can be seen to be very close to 0 which transpire to x_1 having extremely little impact on the model. In terms of the weight attached to the second feature, it can be seen to be a strong negative number, this entails that data points that have larger x_2 values are predicted to be part of the negative class.

$$f(x) = 0.00791148x_1 - 0.41505265x_2 + 0.200817$$

To investigate the effect regularisation has on an SVM model further, the regularisation value chosen for figure 5's visualisation is 1. As seen in the graph, the boundary line constructed follows the same trend as the 0.001 regularisation graph although the boundary line is seen lower on the vertical axis. This weaker level of regularisation allows the model to fit the closer to the true separation boundary as it prioritises the margin size less.

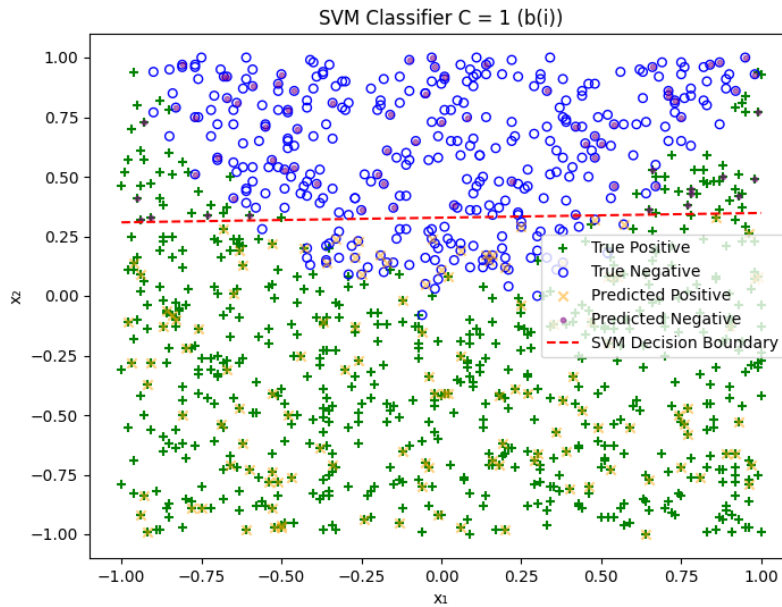


Figure 5: SVM Classifier with a C (regularisation) of 1

The formula represented by the decision boundary can be described by the formula below. Similar to the SVM classifier with 0.001 regularisation, the model assign near to no weight to the x_1 feature which displays that this feature has little to no impact in discriminating between classes. This can be seen visualised on the graph by the fact that both classes have points distributed throughout the x_1 axis without an apparent boundary. The x_2 feature on the other hand is assigned an even stronger negative weight compared to the previous SVM model, this larger negative weight causes the decision boundary to assert itself lower on the vertical axis.

$$f(x) = 0.0357959x_1 - 1.823024x_2 + 0.59848$$

To further investigate the exploration of the effect regularisation on SVM models the C value was increased further to 100, the resulting model can be seen in figure 6 below. Comparing this visualisation to the last model with a regularisation value of 1, the regularisation increase's effect on the model and its decision boundary is very minimal. This demonstrates that after a certain regularisation strength, the effect on the model is negligent as due to its linear nature, the model cannot fit the model any better whilst maintaining an apparent margin between classes. The negligent change between a regularisation strength of 1 and 100 is further expressed by the model's formula seen below which displays minimal difference in the weights assigned.

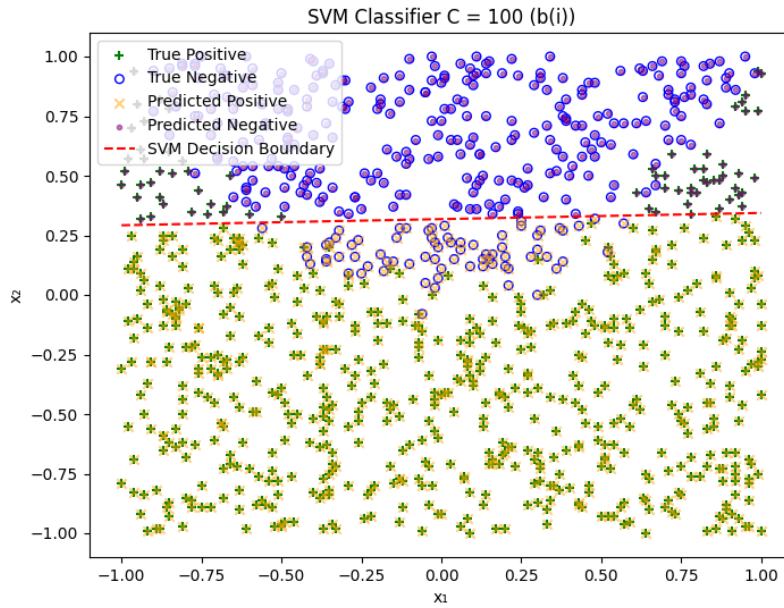


Figure 6: SVM Classifier with a C (regularisation) of 100

$$f(x) = 0.03629x_1 - 1.845856x_2 + 0.60634$$

Comparing these models to the original logistic regression model, the decision boundary lines for the logistic regression, SVM (C=1) and SVM (C=100) are all extremely similar. This can be seen visualised in the graph below which maps each decision boundary on the same graph with the true data points. The only model which does not follow this trend is the first SVM model with a regularisation strength of 0.001. From the graph below it can be inferred that the logistic regression, SVM (C=1) and SVM (C=100) fit the data and will predict the correct class more accurately than the SVM model with 0.001 regularisation as they capture the actual boundary better.

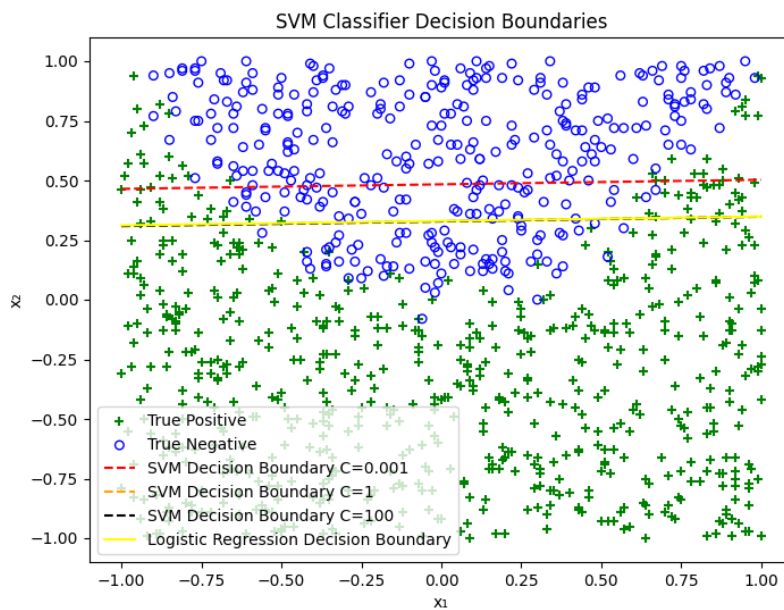


Figure 7: All Model's Decision Boundaries mapped to singular plot with original data

Part C

The final aspect of this investigation entailed taking the original two features and adding the square of each feature to the dataset, resulting in four features in total. With this new feature set, a new logistic regression model was trained resulting in 4 individual weights rather than two. These weights can be seen below assigned to their corresponding feature.

$$p(y = 1|x) = \sigma(0.14251087x_1 - 6.42450608x_2 + 6.15089x_1^2 - 0.7653998x_2^2 + 0.3382)$$

In order to visualise the performance this model can offer, the original data, model predictions and decision boundary were visualised in the graph seen in figure 8. It can be seen from the graph that the model fits the data significantly better compared to the previous models, this is due to the squared features having the ability model the true data boundary more accurately as a quadratic rather than a linear decision boundary.

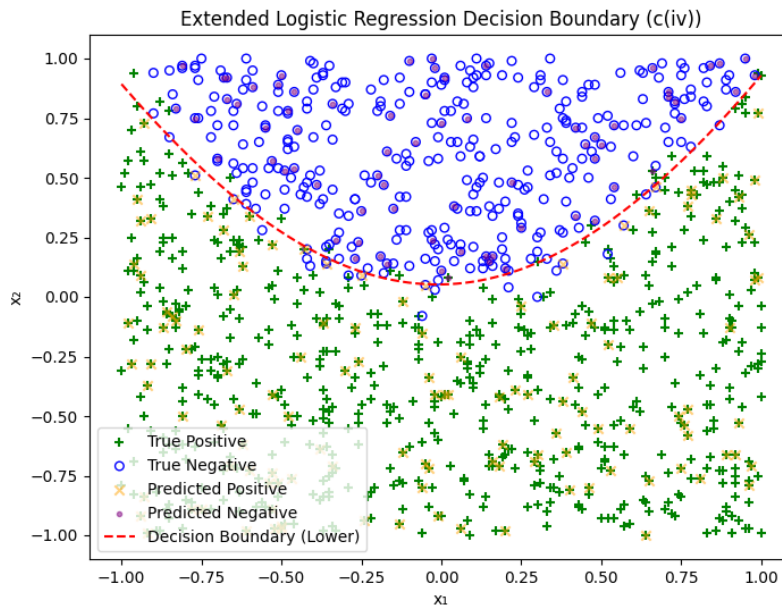


Figure 8: 4 Feature Logistic Regression Model's Decision Boundary and Predicted Points

From the model's formula it was possible to model the decision boundary by solving the quadratic formula identified above. This was done by rearranging the formula as seen below.

$$\omega_3x_2^2 + \omega_1x_2 + (\omega_0x_1 + \omega_2x_1^2 + b) = 0$$

These three components can be assign to “a”, “b” and “c” in the context of the quadratic formula as seen below:

$$a = \omega_3$$

$$b = \omega_1$$

$$c = \omega_0 x_1 + \omega_2 x_1^2 + b$$

From looking at this equation in quadratic for in terms of x_2 , the “c” value is dependent on the value of x_1 . From this the discriminant can be computed, if the discriminant is more than or equal to zero then there is a valid x_2 . Therefore, the discriminant is computed for all x_1 values and the non negative discriminants are stored. Using these valid x_1 values the curved decision boundary can be traced.

In order to compare the performance of this classifier, its accuracy was compared to a baseline predictor as well as the logistic regression model with only 2 features. The accuracy scores achieved by each model can be seen below in figure 9.

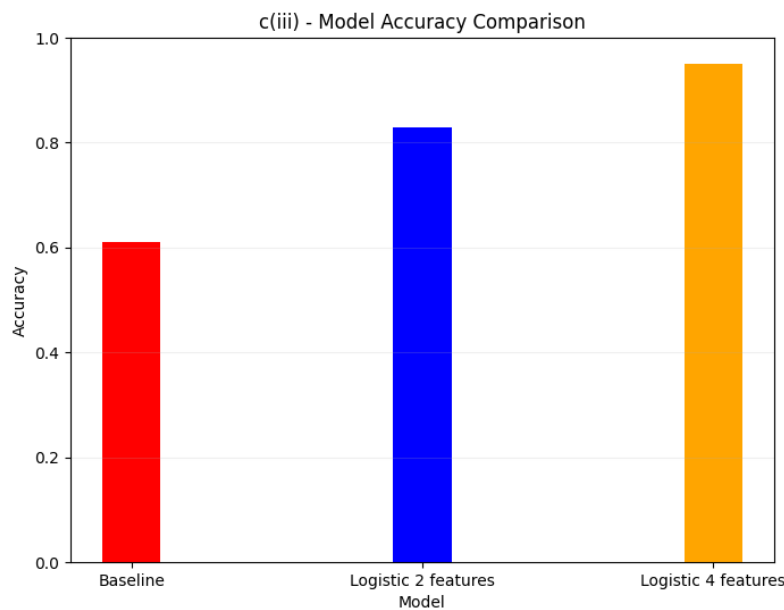


Figure 9: Accuracy of Baseline, 2 Feature Logistic Regression, 4 Feature Logistic Regression Models

Conclusion

To conclude this exercise greatly furthered my knowledge in training logistic regression and SVM models and the effect of different regularisation penalty amounts on a model. As well as this, the exploration into feature engineering was complete by adding two new features by taking the square of each original feature and using all four features to create a 4 feature logistic regression model which proved to have an increased accuracy compared to its 2 feature predecessor.

References

[1] - https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

from sklearn.metrics import accuracy_score

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import StratifiedKFold

#open csv and read first line -. needs to be included in the submission
with open("week2.csv", "r") as f:
    dataset_ID_line = f.readline().strip()
print("Dataset ID line:", dataset_ID_line) # required for submissio

#code given to read the data set
df = pd.read_csv("week2.csv" , comment='#', header=None)
#print(df.head())
X1=df.iloc[:,0]
X2=df.iloc[:,1]
X=np.column_stack((X1 ,X2))
y=df.iloc[:,2]

#a(i) - visulaise data by placing marker on 2D plot
#marker is a plus if the target value is 1 and and a circle if the target value is -1

X1_pos = X1[y == 1]
X1_neg = X1[y == -1]
X2_pos = X2[y == 1]
X2_neg = X2[y == -1]
```

```
#s -s size
```

```
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='Positive')  
plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=20, linewidths=1,  
label='Negative')  
plt.xlabel('X_1')  
plt.ylabel('X_2')  
plt.title('a(i) - 2D Scatter Plot')  
plt.legend()  
plt.show()
```

```
#split data 0.8 training and 0.2 testing
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#a(ii) - train a logistic regression classifier on the data
```

```
#penalty - l2 regularization can discourage large weights but doesn't force them to be zero
```

```
#C -inverse of regularisation - 1 is standard
```

```
model = LogisticRegression(penalty='l2', C=1, solver='liblinear')
```

```
#changed ti o using training set for fitting
```

```
model.fit(X_train, y_train)
```

```
w1, w2 = model.coef_[0] #weights for X1 and X2
```

```
b = model.intercept_[0] #bias term
```

```
#print weights and biases that were learned
```

```
# $p^{\wedge}(y=1|x)=\sigma(b+w_1x_1+w_2x_2)$ 
```

```
# $p^{\wedge}(y=1|x)=1/(1+\exp(-(b+w_1x_1+w_2x_2)))$ 
```

```
print(f'a(ii) - Weights: w1 = {w1}, w2 = {w2}, Bias: b = {b}')
```

```
#changed ti o using test set for accuracy
```

```
y_pred = model.predict(X_test)
```

```
plt.figure(figsize=(8, 6))
```

```
#plot OG data points
```

```
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='True +1')
```

```
plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=30, linewidths=1,  
label='True -1')
```

```
#plot the predicted points
```

```
plt.scatter(X_test[y_pred == 1, 0], X_test[y_pred == 1, 1], marker='x', color='orange',  
label='Predicted +1', alpha=0.5)
```

```
plt.scatter(X_test[y_pred == -1, 0], X_test[y_pred == -1, 1], marker='.', color='purple',  
label='Predicted -1', alpha=0.5)
```

```
#create decision boundary
```

```
# $p(y=1|x) \geq 0.5$  when  $b + wx_1 + wx_2 \geq 0$ 
```

```
x1_range = np.linspace(X1.min(), X1.max(), 100) #100 evenly spaced points between min and max of  
X1
```

```
#solve for x2
```

```
ys = -(w1/w2) * x1_range - (b/w2) #corresponding x2 values for the decision boundary line
```

```
plt.plot(x1_range, ys, color='red', linestyle='--', label='Decision Boundary')
```

```
# Labels
```

```
plt.xlabel("x_1")
```

```
plt.ylabel("x_2")
```

```
plt.title("Logistic Regression Classifier (a(ii) & a(iii))")
```

```
plt.legend()
```

```
plt.show()
```

```
#lets add 5-fold cross validation just to see how it works - will implement properly in next assignment  
;)
```

```

cross_val = StratifiedKFold(n_splits=5,shuffle=True, random_state=42)
log_cross_val_scores = cross_val_score(model, X, y, cv=cross_val,scoring='accuracy')
#print("5-Fold cross validation scores:", {log_cross_val_scores})
print(f'Mean cross validation score: {np.mean(log_cross_val_scores)}')
#a(iii) - plot the training data and the decision boundary

#5 fold cross val for svm - use c = 1 for direct comparison
svm_cross_val_scores = cross_val_score(LinearSVC(C=1, max_iter=10000), X, y, cv=cross_val,
scoring='accuracy')
#print("SVM 5-Fold cross validation scores:", {svm_cross_val_scores})
print(f'SVM Mean cross validation score: {np.mean(svm_cross_val_scores)}')


#b(i) - use sklearn to train svm classifier on the data
#use LinearSVC in sklearn

#train linear svm classifier for a wide range of values and penalties (C=0.001, C=1, C=100)
#give predictions and report parameter values

#change this value as needed to test different penalties
C_values = [0.001, 1, 100]

#train and predict and log the weights and bias for C
print(len(C_values))

#hold all 3 sets of y values for plotting later if needed
#then later we can plot all decision boundaries on the same plot along side the true data points
ys_svm_store = [] #store the y values for each C to plot later if needed

#TO-DO: loop through C_values and train/predict for each
for i in range(len(C_values)):
    svm_model = LinearSVC(C=C_values[i], max_iter=10000)
    svm_model.fit(X_train, y_train)

```

```

y_svm_pred = svm_model.predict(X_test)
w1_svm, w2_svm = svm_model.coef_[0]
b_svm = svm_model.intercept_[0]
print("b(i)SVM Weights: w1 =", w1_svm, "w2 =", w2_svm, "Bias: b =", b_svm)

#accuracy for SVM
svm_acc = accuracy_score(y_test, y_svm_pred)
print("b(i)SVM Accuracy:", svm_acc)

#plot the training and predicted points and decision boundary for SVM
plt.figure(figsize=(8, 6))
#plot OG data points
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='True Positive')
plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=30, linewidths=1,
label='True Negative')

#plot the predicted points
plt.scatter(X_test[y_svm_pred == 1, 0], X_test[y_svm_pred == 1, 1], marker='x', color='orange',
label='Predicted Positive', alpha=0.5)
plt.scatter(X_test[y_svm_pred == -1, 0], X_test[y_svm_pred == -1, 1], marker='.', color='purple',
label='Predicted Negative', alpha=0.5)

#decision boundary for SVM
x1_range = np.linspace(X1.min(), X1.max(), 100)
ys_svm = -(w1_svm/w2_svm) * x1_range - (b_svm/w2_svm)
#test point - check ys_svm values
#print(ys_svm)
ys_svm_store.append(ys_svm) #store for later if needed
plt.plot(x1_range, ys_svm, color='red', linestyle='--', label='SVM Decision Boundary')
# Labels
plt.xlabel("x_1")
plt.ylabel("x_2")
plt.title("SVM Classifier C = " + str(C_values[i]) + " (b(i))")
plt.legend()
plt.show()

```

```

#plot all decision boundaries on the same plot along side the true data points
plt.figure(figsize=(8, 6))

#plot OG data points
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='True Positive')
plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=30, linewidths=1,
label='True Negative')

colors = ['r', 'orange', 'black']

#plot decision boundaries for each C
for i in range(len(C_values)):

    #need more distinct line colors/styles if plotting all 3

    #make array of strings containing colors and iterate through at the same time as C_values
    plt.plot(x1_range, ys_svm_store[i], linestyle='--', label='SVM Decision Boundary C=' +
str(C_values[i]), color=colors[i])

#now plot the logistic regression decision boundary for comparison
ys_log = -(w1/w2) * x1_range - (b/w2)

plt.plot(x1_range, ys_log, color='yellow', linestyle='-', label='Logistic Regression Decision
Boundary')

# Labels
plt.xlabel("x_1")
plt.ylabel("x_2")

plt.title("SVM Classifier Decision Boundaries")

plt.legend()

plt.show()

#c(i) - create two additional features a=> add square of each feature (four features in total). Train
logistic classifier give the model and the trained parameters values

#create squared features for training and testing sets
X1_squared_train = X_train[:,0]**2
X2_squared_train = X_train[:,1]**2
X1_squared_test = X_test[:,0]**2
X2_squared_test = X_test[:,1]**2

```

```

#stack to create extended feature sets

X_ext_train = np.column_stack((X_train[:,0], X_train[:,1], X1_squared_train, X2_squared_train))
X_ext_test = np.column_stack((X_test[:,0], X_test[:,1], X1_squared_test, X2_squared_test))


#train log reg model on training set with extended features
model_ext = LogisticRegression(penalty='l2', C=1, solver='liblinear')
model_ext.fit(X_ext_train, y_train)


w = model_ext.coef_[0] #weights for X1, X2, X1^2, X2^2
b_ext = model_ext.intercept_[0] #bias term

print(f'c(i) - extended logistic regression weights: w1 = {w[0]}, w2 = {w[1]}, w3 = {w[2]}, w4 = {w[3]}, Bias: b = {b_ext}')

y_ext_pred = model_ext.predict(X_ext_test)


#plot the true data points and the predicted points
plt.figure(figsize=(8, 6))

#plot OG data points
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='True Positive')

plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=30, linewidths=1,
label='True Negative')

#plot the predicted points
plt.scatter(X_test[y_ext_pred == 1, 0], X_test[y_ext_pred == 1, 1], marker='x', color='orange',
label='Predicted Positive', alpha=0.5)

plt.scatter(X_test[y_ext_pred == -1, 0], X_test[y_ext_pred == -1, 1], marker='.', color='purple',
label='Predicted Negative', alpha=0.5)

# Labels
plt.xlabel("x_1")
plt.ylabel("x_2")

plt.title("Logistic Regression Classifier with Square of Each Feature (c(i))")

plt.legend()

plt.show()


#c(iii) compare performance with reasonable baseline predictor

```

```

#baseline - predict the most frequent class in the training set

#y_train is all +1 and -1 so if the sum of +1 is greater than sum of -1 then its the most frequent class
#use >= in case they are equal
if(np.sum(y_train == 1) >= np.sum(y_train == -1)):
    most_freq_class = 1
else:
    most_freq_class = -1

baseline_pred = np.full(y_test.shape, most_freq_class)

#calculate accuracy for baseline logistic regression and extended logistic regression
baseline_acc = accuracy_score(y_test, baseline_pred)
logistic_acc = accuracy_score(y_test, model.predict(X_test))
logistic_ext_acc = accuracy_score(y_test, model_ext.predict(X_ext_test))


#print accuracy values
print("c(iii) - Baseline accuracy:", baseline_acc)
print("c(iii) - Logistic regression (2 features) accuracy:", logistic_acc)
print("c(iii) - Logistic regression (4 features) accuracy:", logistic_ext_acc)


#plot bar graph to compare accuracies
labels = ['Baseline', 'Logistic 2 features', 'Logistic 4 features']
accuracies = [baseline_acc, logistic_acc, logistic_ext_acc]
x = np.arange(len(labels))
width = 0.2

plt.figure(figsize=(8, 6))
plt.bar(x, accuracies, width, color=['red', 'blue', 'orange'])
plt.xticks(x, labels)
plt.ylim(0, 1)
plt.ylabel('Accuracy')
plt.xlabel('Model')
plt.title('c(iii) - Model Accuracy Comparison')
plt.grid(axis='y', alpha=0.2)
plt.show()

```

```

#c(iv) - bonus - plot the classifier boundary
#have to solve a quadratic equation to get the decision boundary
x1_range = np.linspace(X1.min(), X1.max(), 400)
a = w[3] #a component
b = w[1] #b component
c = w[0]*x1_range + w[2]*x1_range**2 + b_ext
discriminant = b**2 - 4*a*c
#only consider points where discriminant is non-negative
non_neg = discriminant >= 0
x1_non_neg = x1_range[non_neg]
sqrt_discrim = np.sqrt(discriminant[non_neg])
x2_pos = (-b + sqrt_discrim) / (2*a)
x2_neg = (-b - sqrt_discrim) / (2*a)
#plot the decision boundary along with data points
plt.figure(figsize=(8, 6))
#plot OG data points
plt.scatter(X1_pos, X2_pos, marker='+', color='g', label='True Positive')
plt.scatter(X1_neg, X2_neg, marker='o', facecolors='none', edgecolors='b', s=30, linewidths=1,
label='True Negative')
#plot the predicted points
plt.scatter(X_test[y_ext_pred == 1, 0], X_test[y_ext_pred == 1, 1], marker='x', color='orange',
label='Predicted Positive', alpha=0.5)
plt.scatter(X_test[y_ext_pred == -1, 0], X_test[y_ext_pred == -1, 1], marker='.', color='purple',
label='Predicted Negative', alpha=0.5)
#plot decision boundary
#plt.plot(x1_valid, x2_pos, color='red', linestyle='--', label='Decision Boundary (Upper)')
plt.plot(x1_non_neg, x2_neg, color='red', linestyle='--', label='Decision Boundary (Lower)')
# Labels
plt.xlabel("x_1")
plt.ylabel("x_2")
plt.title("Extended Logistic Regression Decision Boundary (c(iv))")
plt.legend()

```

```
plt.show()
```